

Introduction to Python Programming

Chapter 0: Welcome to the Course

© 2025 Iles W. Wade

License: This material is free to use for educational and personal purposes. You may share, copy, and adapt this material provided that:

- You do not create or sell commercial products derived from this material
- You do not claim exclusive copyright over this material
- You provide attribution to the original author

Created: October 2025

About This Course

Welcome to Introduction to Python Programming!

Important Note: While this manual covers a comprehensive range of topics from fundamental programming concepts to advanced object-oriented programming, you are not necessarily expected to cover everything in this manual for your course. Some topics, particularly in the later chapters and appendices, have been included as future reference material for continued learning beyond this course. **Please check with your instructor about which specific chapters and topics are relevant to your current course requirements.**

This course uses Python as a vehicle to teach you **object-oriented programming concepts** that extend far beyond any single language. While we'll be working specifically with Python and Visual Studio Code, our real goal is to help you understand the fundamental principles of programming that you can apply to any language you encounter in your career.

Important Philosophy: Python is an excellent starting point for learning programming, but don't get too attached to the syntax. The concepts we cover—variables, control structures, functions, classes, and object-oriented design—exist in virtually every modern programming language. We could rewrite this entire manual for Rust, Java, or C++, and while the syntax would change (lists might become arrays or vectors), the underlying concepts would remain the same.

Keep your focus on **how to think like a programmer** rather than just memorizing Python syntax. Understanding the "why" behind programming concepts will serve you far better than just knowing the "how" in one specific language.

Course Tools

Visual Studio Code (VS Code): Your code editor for the course. Remember, VS Code is NOT Python—it's a tool for writing code in many languages.

Python: The programming language we're using. We chose Python because it has clear, readable syntax that helps you focus on concepts rather than fighting with complicated syntax rules.

How to Use This Manual

This manual is structured to build your knowledge progressively. Each chapter introduces new concepts that build on what you've learned before. Don't skip ahead—the foundation matters!

Recommendations:

- **Read sequentially** - Later chapters assume knowledge from earlier ones
 - **Type the code yourself** - Don't just read examples, write them
 - **Do the practice exercises** - They reinforce what you've learned
 - **Answer reflection questions honestly** - They help identify gaps in understanding
-

Chapter Guide

Chapter 0: Course Introduction

Core Concept: Course Philosophy and Structure

This chapter! An introduction to the course philosophy, tools, and structure. You'll understand why we focus on concepts over syntax, how the course is organized, and what you'll learn in each chapter.

Chapter 1: Getting Started with Programming

Core Concept: Algorithmic Thinking

This chapter introduces what programming actually is—problem solving through algorithms. You'll learn that programming isn't about memorizing syntax; it's about breaking problems down into logical steps. We cover the difference between interpreted and compiled languages, set up your development environment (Python and VS Code), and establish good organizational habits with proper folder structures. The key takeaway: thinking algorithmically is more important than knowing any specific language.

Key Topics:

- What is an algorithm?
- Python as an interpreted language
- Setting up Python and VS Code

- Understanding file paths and project organization
 - The relationship between problem-solving and code
-

Chapter 2: Python Basics

Core Concept: Building Blocks of Code

Here you'll learn Python's fundamental building blocks. We cover critical syntax rules (like why white space matters in Python), naming conventions (snake_case for variables, UPPERCASE for constants), and the four basic data types (integers, floats, strings, and booleans). You'll understand dynamic typing, perform mathematical operations, and learn type casting—including the important concept of lossy conversions. Comments and documentation practices are also introduced.

Key Topics:

- White space dependency in Python
 - Variable naming conventions (snake_case)
 - Keywords and reserved words
 - Data types: int, float, str, bool
 - Dynamic typing
 - Mathematical operators
 - Type casting and conversion
 - Comments and docstrings
-

Chapter 3: Input and Output - Introduction

Core Concept: Program Interaction

This chapter makes your programs interactive! You'll learn how to display information with `print()` and gather input with `input()`. A critical concept here: `input()` ALWAYS returns a string, so you must cast user input for mathematical operations. We cover three methods of formatting output (concatenation, commas, and f-strings), with f-strings being the recommended modern approach. You'll also learn about the limitations of input validation at this stage and why programs can crash with unexpected input.

Key Topics:

- The `print()` function and its versatility
- String quotes (single vs double)
- Concatenation vs commas vs f-strings
- The `input()` function and its string nature
- Type casting user input

- Why input validation matters (preview)
 - Escape characters
 - Building simple interactive programs
-

Chapter 4: Input and Output - Intermediate

Core Concept: Professional Output Formatting

This chapter elevates your output from basic to professional. You'll master the `end` and `sep` parameters to control print behavior, learn advanced f-string formatting for precise decimal places, alignment, and width control. Raw strings are introduced for handling file paths and special characters. These techniques are essential for creating well-formatted reports, tables, and user interfaces.

Key Topics:

- The `end` parameter and same-line printing
 - The `flush` parameter for real-time output
 - The `sep` parameter for custom separators
 - F-string formatting specifications
 - Number formatting (decimals, width, alignment)
 - Thousands separators and percentages
 - Raw strings for file paths
 - Creating formatted tables and reports
-

Chapter 5: If Statements and Conditions

Core Concept: Decision-Making and Code Blocks

This is where programs become "smart." If statements allow your code to make decisions and respond dynamically to different conditions. The chapter begins with a crucial concept: **code blocks and indentation**. Understanding that indentation creates logical groupings of code is fundamental not just for if statements, but for loops, functions, and classes you'll encounter later. You'll learn comparison operators, logical operators (and, or, not), and how to build complex decision trees using if-elif-else chains and nested if statements.

Key Topics:

- Code blocks and indentation (critical concept!)
- Comparison operators (`==`, `!=`, `>`, `<`, `>=`, `<=`)
- Basic if statements
- If-else for two-way decisions

- If-elif-else for multiple options
 - Logical operators: and, or, not
 - Nested if statements
 - Order of evaluation
 - Building complex decision logic from simple pieces
 - Match statements (optional, modern Python)
-

Chapter 6: Loops

Core Concept: Repetition and Automation

This chapter introduces one of programming's superpowers: the ability to repeat actions automatically. You'll learn two types of loops—for loops (when you know how many iterations you need) and while loops (when repetition depends on a condition). Understanding `range()` is key here, especially why it starts at 0 and how it works perfectly with list indexing. The chapter covers `break` and `continue` statements for controlling loop flow, and importantly, nested loops for handling multi-dimensional problems. This is where automation really begins—instead of writing code 100 times, write it once and loop it 100 times!

Key Topics:

- For loops vs while loops (when to use each)
 - The `range()` function and its parameters
 - Why range starts at 0 (computer science foundations)
 - While loops and condition-based repetition
 - Break statement (exit loop immediately)
 - Continue statement (skip to next iteration)
 - The `while True` pattern with `break`
 - Nested loops and their behavior
 - Common loop patterns (accumulator, counter, validation)
-

Chapter 7: Lists - Introduction

Core Concept: Data Structures and Collections

Welcome to your first data structure! Lists allow you to store multiple values in a single variable—a fundamental concept in programming. This chapter explains zero-based indexing (and why it matters from a memory perspective), how to access elements, and the critical relationship between list length and the last index. You'll learn to iterate through lists using both range-based loops and the cleaner for-each approach. The chapter also covers the `enumerate()` function for when you need both the index and value. A crucial insight:

strings behave like lists of characters, so everything you learn here applies to string manipulation too. Understanding lists is foundational—virtually every program you write will use them.

Key Topics:

- What is a data structure?
 - Creating and accessing lists
 - Zero-based indexing explained
 - Negative indexing (counting from end)
 - List length and the last element relationship
 - Range-based iteration vs for-each loops
 - The `enumerate()` function
 - Naming convention: plural list names, singular loop variables
 - Strings as character lists
 - Mutable vs immutable (lists vs strings)
 - Nested loops with lists
-

Chapter 8: Lists - Intermediate

Core Concept: Mutability and Memory Management

This chapter dives deeper into how lists actually work in memory—a critical concept that trips up many beginners. The most important lesson: assignment creates a reference, not a copy. When you write `list_b = list_a`, both variables point to the SAME list in memory. This chapter thoroughly explains the difference between references and copies, and when to use each. You'll learn all the methods for modifying lists: adding elements (`append`, `insert`, `extend`), removing elements (`remove`, `pop`, `del`, `clear`), and finding elements (`in` operator, `index()`, `count()`). List slicing is introduced as a powerful tool for extracting portions of lists, and it works identically on strings. Understanding mutability and references is crucial as you move toward functions and object-oriented programming.

Key Topics:

- Mutability: lists can change, strings cannot
- References vs copies (critical memory concept)
- Assignment creates references, `.copy()` creates copies
- Adding elements: `append()`, `insert()`, `extend()`
- Removing elements: `remove()`, `pop()`, `del`, `clear()`
- Finding elements: `in` operator, `index()`, `count()`
- List slicing: `[start:stop:step]`

- Slicing with negative indices
 - String slicing (works like lists)
 - Sorting and reversing lists
 - Memory visualization and why this matters
-

Chapter 9: Exceptions and Error Handling

Core Concept: Graceful Failure and Program Robustness

Up until now, errors meant your program crashed. Professional programmers expect errors and handle them gracefully—that's what this chapter teaches. You'll learn about Python's exception types (`ValueError`, `TypeError`, `ZeroDivisionError`, etc.) and how to catch them using try-except blocks. The chapter covers the full try-except-else-finally structure, showing when each part runs and why it matters. Exception handling is essential for user input validation, file operations, and any situation where things might go wrong. The chapter also introduces context managers and the `with` statement—a preview of automatic resource management that you'll use constantly for file operations. Understanding the exception hierarchy helps you catch the right errors at the right level of specificity.

Key Topics:

- What are exceptions and when do they occur?
 - Common exception types and what triggers them
 - Try-except blocks for handling exceptions
 - Catching specific vs general exceptions
 - The else clause (runs when no exception occurs)
 - The finally clause (always runs)
 - Accessing exception details
 - Raising your own exceptions
 - Exception hierarchy and inheritance
 - Why order matters when catching exceptions
 - Context managers and the with statement (preview)
 - Best practices: catch specific, keep try blocks small
-

Chapter 10: File Input/Output

Core Concept: Persistent Data and External Resources

Programs that can't save data are severely limited. This chapter teaches you how to read from files and write to files, making your programs capable of storing information between runs. You'll learn file modes (read, write, append), how to use the `with` statement for automatic file closing (that context manager from Chapter 9!), and various methods for reading (`read()`, `readline()`, `readlines()`). The chapter covers both text and binary files, error handling for file operations (combining with Chapter 9's exception handling), and working with file paths using `os.path`. JSON is introduced as a structured data format, and you'll learn common patterns like reading-processing-writing and safe file updates with backups. Understanding file I/O is essential for real-world applications—configuration files, logs, data storage, and user content all depend on these skills.

Key Topics:

- What is File I/O and why it matters
 - File paths: absolute vs relative
 - File modes: 'r', 'w', 'a', and their variants
 - Reading files: `read()`, `readline()`, `readlines()`
 - Writing and appending to files
 - The `with` statement for automatic cleanup
 - Exception handling for file operations
 - Working with file paths using `os.path`
 - Binary files and file copying
 - JSON for structured data storage
 - Common file operation patterns
 - Best practices: always use `with`, handle exceptions
-

Chapter 11: Functions - Introduction

Core Concept: Code Reusability and Organization

Functions are mini-programs within your program that perform specific tasks. This chapter teaches you to write code once and reuse it many times, dramatically reducing repetition and making debugging easier. You'll learn the critical distinction between defining a function (creating the template) and calling it (running the code), and why functions must be defined before they're used. The chapter covers parameters vs arguments—a subtle but important difference—and introduces return values, showing how functions send data back. You'll learn to write functions that return boolean values both verbosely and compactly (`return condition` instead of `if/else` blocks). Understanding functions is essential for organizing code into logical, reusable pieces.

Key Topics:

- What functions are and why they're essential
- Function definition vs function call
- Definition order matters (define before use)

- Parameters (placeholders) vs arguments (actual values)
 - Return values vs print statements
 - Boolean return patterns (compact vs verbose)
 - Conditional returns and early exits
 - Edge case handling in functions
 - Common function patterns
-

Chapter 12: Functions - Intermediate

Core Concept: Flexibility and Recursion

This chapter elevates your function skills with advanced techniques. You'll learn about positional vs named arguments—how to call functions with explicit parameter names for clarity. Default parameters are introduced, showing how to make arguments optional, which is Python's elegant alternative to function overloading found in languages like Java or C++. The chapter then introduces recursion: functions that call themselves. Through examples like countdown, Fibonacci, and factorial, you'll understand the three essential components every recursive function needs: a base case (stopping condition), recursive case (calling itself), and progress toward the base case. You'll also learn about the dangerous pitfall of mutable default arguments and why to avoid them. These techniques make your functions more flexible and capable of solving complex problems.

Key Topics:

- Positional vs named (keyword) arguments
 - Mixing positional and named arguments (rules)
 - Default parameters for optional arguments
 - Default parameters must come last
 - Python's approach vs function overloading
 - Introduction to recursion
 - Base case, recursive case, progress toward base
 - Fibonacci sequence as recursive example
 - Recursion vs iteration (when to use each)
 - Mutable default argument pitfall
-

Chapter 13: Data Structures - Intermediate

Core Concept: Choosing the Right Container

Beyond lists, Python offers three more essential data structures, each optimized for specific tasks. Tuples are immutable sequences—perfect for data that shouldn't change, like GPS coordinates or RGB colors. They support unpacking, allowing elegant variable assignment and swapping. Sets store unique values with blazingly fast membership testing, automatically removing duplicates and supporting mathematical operations (union, intersection, difference). Dictionaries store key-value pairs for fast lookup by meaningful names rather than numeric indices. This chapter teaches you when to choose each structure: lists for ordered mutable collections, tuples for immutable sequences, sets for unique items and fast lookup, and dictionaries for key-value relationships. Understanding these structures and their tradeoffs is fundamental to writing efficient Python code.

Key Topics:

- Tuples: immutable sequences
 - Tuple unpacking and variable swapping
 - When to use tuples vs lists
 - Sets: unique unordered collections
 - Set operations: union, intersection, difference
 - Fast membership testing with sets
 - Dictionaries: key-value pairs
 - Dictionary methods: keys(), values(), items()
 - Safe dictionary access with get()
 - Comparing all data structures
 - When to use each structure
-

Chapter 14: Data Structures - Intermediate Concepts

Core Concept: Complex Data Organization

Real-world data is rarely simple—it's hierarchical, nested, and interconnected. This chapter explores nested data structures: lists within dictionaries, dictionaries within lists, and more complex combinations. You'll learn to work with 2D lists (matrices), nested dictionaries for hierarchical data like API responses, and mixed structures for maximum flexibility. The chapter introduces JSON (JavaScript Object Notation), the standard format for web data exchange, showing how Python dictionaries map naturally to JSON. A practical section covers GPS coordinates using tuples, including the Haversine formula for calculating distances between coordinates and bearing calculations for finding points at specific distances and directions. Understanding nested structures and JSON is essential for working with APIs, configuration files, and complex data models.

Key Topics:

- Nested data structures (lists in dicts, dicts in lists)
- 2D lists and matrices

- Frozensets (immutable sets)
 - JSON: JavaScript Object Notation
 - `json.dumps()` and `json.loads()` for serialization
 - Saving and loading JSON files
 - Python-JSON type mappings
 - GPS coordinates with tuples
 - Haversine distance formula
 - Bearing calculations for navigation
 - Practical applications: APIs, config files
-

Chapter 15: Classes - Introduction

Core Concept: Templates for Objects (OOP Begins)

This is where object-oriented programming truly begins. The chapter opens with a thought-provoking question: "There is no such thing as a cow"—exploring the difference between a template (the general concept "cow") and instances (specific cows like Betsy or Bessie). This philosophical foundation leads to classes: templates that define what something is and can do. You'll learn that a class itself isn't a usable thing—objects (instances) created from the class are the real entities in your program. The `__init__` method (initializer) is introduced for setting up new objects, and instance variables store data specific to each object. The chapter emphasizes that `self` represents "this specific object" and shows how one template can generate unlimited unique objects, each with its own data. This is the conceptual foundation of OOP—understanding the template-instance relationship is crucial for everything that follows.

Key Topics:

- Philosophy: template (class) vs instances (objects)
 - Classes as templates, objects as instances
 - Creating classes with `class ClassName:`
 - The `__init__` initializer method
 - Instance variables: `self.variable_name`
 - Understanding `self` (represents this object)
 - Creating multiple objects from one class
 - Accessing object data with dot notation
 - Why classes matter: organization, reusability, modeling
 - Real-world examples: books, cars, students
-

Chapter 16: Classes - Intermediate

Core Concept: Methods and Object Behavior

Objects don't just store data—they also perform actions. This chapter introduces methods: functions that belong to classes and can access/modify object state. You'll learn two complementary ways to understand `self`: as "this specific object" and as the automatically-passed first argument to every method. The concept of state (all the current values of an object's instance variables) is explored, showing how methods read, change, and use state to make decisions. Special methods `__str__` and `__repr__` are introduced—these customize how objects appear when printed or represented. Understanding that methods have access to `self` and thus to all instance data is crucial. This chapter also notes that while Python uses `self`, other languages use `this` for the same concept.

Key Topics:

- Methods: functions inside classes
 - Understanding `self` (two perspectives)
 - How Python passes objects to methods automatically
 - Object state: all instance variable values at any moment
 - Methods that return values
 - `__str__` for human-readable output (used by `print`)
 - `__repr__` for developer-friendly representation
 - Preview of other special methods (`__len__`, `__add__`, etc.)
 - Methods vs functions: access to object state
 - The `this` keyword in other languages
-

Chapter 17: Classes - Advanced

Core Concept: Professional Class Design Patterns

This chapter covers advanced techniques that professional developers use daily. Class variables (shared by all instances) are contrasted with instance variables (unique to each object), showing when to use each. Static methods (`@staticmethod`) are utility functions that belong to the class but don't access instance or class data. Class methods (`@classmethod`) receive the class itself as an argument and are perfect for alternative constructors or factory methods. Enumerations provide type-safe named constants, preventing invalid values. The Singleton pattern ensures only one instance of a class exists—useful for database managers or configuration systems. Nested classes group helper classes within their parent. The chapter also explains `__new__` (which creates objects before `__init__` initializes them), essential for implementing Singletons. These patterns solve real-world problems and represent professional Python development.

Key Topics:

- Class variables vs instance variables
- Static methods: utility functions in class namespace
- Class methods: alternative constructors and factories
- The `cls` parameter in class methods
- Enumerations (Enum) for named constants
- Auto-numbering enums with `auto()`
- Singleton pattern for single-instance classes
- `__new__` vs `__init__` (creation vs initialization)
- Nested classes for helper classes
- Thread-safe Singleton implementation
- When to use each pattern
- Combining advanced concepts

Course Complete: From Foundations to Professional Development

You've now been introduced to the complete curriculum—from basic variables through advanced object-oriented programming.

What You've Learned:

Foundations (Chapters 0-10): Introduction to programming concepts, variables, control flow, loops, lists, error handling, and file I/O—the building blocks every programmer needs.

Functions (Chapters 11-12): Code organization, reusability, recursion, and flexible parameter handling—tools for managing complexity.

Data Structures (Chapters 13-14): Tuples, sets, dictionaries, nested structures, and JSON—choosing the right container for your data.

Object-Oriented Programming (Chapters 15-17): Classes as templates, objects as instances, methods for behavior, and professional patterns—the paradigm that dominates modern software development.

Supplementary Appendices: Additional resources covering Python's library ecosystem, advanced OOP concepts like inheritance and polymorphism, and functional programming patterns—available for deeper exploration as your projects demand.

The Core Philosophy Realized:

These concepts transcend Python. You've learned universal programming principles that apply to Java, C++, C#, Rust, and beyond:

- Classes create objects (template → instance)

- Functions encapsulate logic
- Data structures organize information
- Patterns solve recurring problems
- Libraries extend capabilities
- Inheritance models relationships
- Polymorphism provides flexibility

Whether you write `self` (Python) or `this` (Java/JavaScript/C++), whether you use inheritance or interfaces, whether you prefer imperative or functional style—the **fundamental concepts** remain constant. The syntax is temporary; understanding is permanent.

What's Next:

With this foundation, you can:

- Build substantial Python applications using OOP principles
- Learn other OOP languages more easily (concepts translate directly)
- Recognize and apply design patterns to solve common problems
- Explore the appendices for practical library usage and advanced topics
- Make informed decisions about code architecture and design

The core chapters (0-17) represent essential knowledge every programmer needs. The appendices provide practical tools and advanced concepts for continued learning and professional development. Everything builds on the foundation—master the basics, then explore the advanced topics as your projects demand.

Congratulations on completing this comprehensive journey from programming fundamentals to professional development patterns!